NERDD

REST API: CYPstrate

CYPstrate consists of a collection of machine learning classifiers (random forest and support vector machines) for the prediction of substrates and non-substrates of the nine most important human CYP isozymes in the metabolism of xenobiotics (i.e. CYPs 1A2, 2A6, 2B6, 2C8, 2C9, 2C19, 2D6, 2E1 and 3A4). The models are trained on a high-quality data set of 1831 substrates and non-substrates compiled from public sources. Two distinct prediction modes are available to cover different use cases.

This web service is freely available for both academic and commercial use.

Quickstart

Creating a computational job for the module **CYPstrate** and retrieving the results can be done in a few simple steps:

Python

```python
import time, requests

# create job
response = requests.post(
```

```
        "https://nerdd.univie.ac.at/api/cypstrate/jobs",
        data={
            "inputs": ["CN1C=NC2=C1C(=O)N(C(=O)N2C)C"],
            "prediction_mode": "best_performance",
        },
    ).json()
    job_id = response["id"]

    # wait for job to finish
    while True:
        response = requests.get(
            f"https://nerdd.univie.ac.at/api/jobs/{job_id}
        ).json()
        if response["status"] == "completed":
            break
        time.sleep(5)

    # display results
    response = requests.get(
        f"https://nerdd.univie.ac.at/api/jobs/{job_id}/res
    ).json()
    for result in response["data"]:
        print(result)
```

Re

Create a simple job

You can create small jobs for testing / demonstration purposes by sending GET requests to the `/cypstrate/jobs` endpoint. Use the parameter `inputs` to provide input molecules in common representations (e.g. SMILES, InChI). You can specify more than one input molecule by setting `inputs` multiple times (see code below). The number of input molecules is limited, because the length of the resulting URL is constrained by web browsers, client libraries, and web servers. If you need to process more input molecules, you should use a POST request instead (see below, Large jobs).

You can specify job parameters by adding them as GET parameters. Check the module's endpoint `/cypstrate/jobs` in the API

documentation for additional information about the names and types of all parameters available.

Python

```python
response = requests.get(
    # this is a single URL string
    "https://nerdd.univie.ac.at/api/cypstrate/jobs"
    "?inputs=CN1C=NC2=C1C(=O)N(C(=O)N2C)C"
    "&inputs=CCO"
    "&prediction_mode=best_performance"
).json()
```

Large jobs

For larger jobs, you need to use a POST request to the `/cypstrate/jobs` endpoint and provide the input molecules in the request body.

Python

```python
response = requests.post(
    "https://nerdd.univie.ac.at/api/cypstrate/jobs",
    data={
        "inputs": ["CN1C=NC2=C1C(=O)N(C(=O)N2C)C"],
        "prediction_mode": "best_performance",
    }
).json()
```

Upload an input file

To upload an input file, you need to use a POST request to the `/cypstrate/jobs` endpoint and include the file in the request body.

Note: files can not be uploaded in the corresponding GET endpoint. You have to use a POST request.

Python

```python
with open("/path/to/my/input_file.smiles", "r") as f:
    response = requests.post(
        "https://nerdd.univie.ac.at/api/cypstrate/jobs
        files={"files": f}
    ).json()
```

Get job status

After creating a job, you can check its status by sending a GET request to the job's endpoint. The response contains information about the job's progress (`status`, `num_entries_processed`, `num_entries_total`, `num_pages_processed`, `num_pages_total`).

Python

```python
response = requests.get(
    f"https://nerdd.univie.ac.at/api/jobs/{job_id}"
).json()
```

Get results

Results are served in multiple pages containing a fixed number of entries each. The number of total pages can be determined by the `num_pages_total` field in the job status response (see above) or by fetching the results of the first page (see below) and reading the `job.num_pages_total` field. Note that the `num_pages_total` field is undefined as long as the number of input molecules is not known, yet. The request has to be repeated (at sensible intervals) until the `num_pages_total` field is set.

The `page` parameter can be used to iterate through the pages. Note that the first page is `page=1`, not `page=0`.

Python

```python
response = requests.get(
    f"https://nerdd.univie.ac.at/api/jobs/{job_id}/res
```

```
).json()
```

Delete job

Jobs can be deleted by sending a DELETE request to the job's endpoint.

Python

```
response = requests.delete(
    f"https://nerdd.univie.ac.at/api/jobs/{job_id}"
).json()
```

Additional notes

- When adding a trailing slash to a request URL (e.g. `/jobs/123/` intead of `/jobs/123`), the server will redirect to the non-slash version using a 307 redirect response. Some libraries like `requests` in Python or the function `fetch` in JavaScript will follow this redirect automatically, while others like `curl` will need to be configured to do so.
- Result pages are indexed starting from 1 to be consistent with the web UI. The first page is `page=1` and accessing page 0 will return an error (status 404, Not Found).
- Result pages can be fetched using the query parameter `incomplete` (e.g. `/jobs/123/results?page=1&incomplete=true`). This will return all results that are available at the time of the request, even if not all results on the requested page are processed, yet. The response contains a field `pagination.is_incomplete` that will be set to `true` if the page is incomplete and more results are expected to be available later.
- A request for a non-existing result page might return a 202 (Accepted) response if the job is still running and the requested page exists but is not yet available. However, a 404 (Not Found) response indicates that the page is out of range (or the job has been deleted).

Full API documentation

See all endpoints at https://nerdd.univie.ac.at/api/docs.

NERDD

Next-generation E-Resource for Drug Discovery

NERDD is free for both academic and commercial use. Some models have commercial-use restrictions due to their training data, which are clearly noted on each model's page.

Developed by Steffen Hirte

About

- COMP3D Group
- Privacy Policy
- Legal Notice
- Contact

Links

- Documentation
- Developer API
- How to cite

Code

- Architecture
- Frontend
- Backend
- NERDD Module

Partners

universität wien

Universität Hamburg
DER FORSCHUNG | DER LEHRE | DER BILDUNG